

Polymarket Price Tracker

Version 1 — Implementation Specification

Prepared for implementation with Claude Code. This document consolidates the client's job posting, all clarifications from the client conversation, the approved frontend template, verified Polymarket API details, and the agreed technical decisions into a single build-ready brief.

Developer	Melek
Date	July 15, 2026
Timeline quoted	3–4 days
Approved artifact	polymarket-tracker-dashboard.jsx (React mockup, client has seen screenshots)
Constraints	Open-source only, no paid libraries, no Docker, runs 24/7 on Ubuntu, SQLite storage

1. Project Overview

The client wants a self-hosted web application that collects and stores historical price data from Polymarket prediction markets, running continuously on his Linux server. His own words: “My interest is the data. If you can track and save the price point for whatever I track.” The browser UI is a viewer and control panel; the collector must run server-side and keep polling with the browser closed.

The client explicitly requested a React frontend template that he can later build his own backend variations on, with a plain white background and a visible CSV export button. He has approved the look of the current template. This is Version 1 only; he plans to expand the project, so clean, modular, commented code is a hard requirement.

1.1 Decisions agreed during the client conversation

The following points were raised by the client after the initial proposal and are now part of the scope. First, opening a market must show both the interactive chart and a chronological time-series table with one row per poll (timestamp plus the price of every outcome), so he can “replay” the market tick by tick. Second, adding something to track works at the event level: the user pastes a Polymarket event URL or ID, the app lists every market (prop) inside that event — match winner, totals such as Over/Under 2.5 goals, both teams to score, and so on — and the user ticks which props to track. Each tracked prop becomes its own independent price series with its own history and CSV export. Third, polling every second is technically possible within Polymarket's limits, but 3–5 seconds per market was communicated to the client as the sensible default; the interval is configurable per market. Fourth, the chart library is Recharts rather than the Plotly mentioned in the original cover letter — it is the natural fit for a React template the client will extend himself.

1.2 Out of scope for V1 (documented upsells)

No trading, no wallets, no authentication against Polymarket (read-only public data needs none). No WebSocket streaming — the CLOB WebSocket feed is the V2 path if the client later wants true sub-second data. No backfill of history from before tracking started, although the CLOB /prices-history endpoint makes this an easy V2 feature. No user accounts or multi-user support.

2. Technology Stack

Layer	Choice	Notes
Language	Python 3.11+	Backend and tooling
Web framework	FastAPI + Uvicorn	Serves REST API and the built frontend bundle
Scheduler	APScheduler	One polling job per tracked market, interval configurable per market; runs in-process, independent of any browser
Database	SQLite	Single prices.db file; WAL mode for concurrent read (API) + write (collector)
HTTP client	httpx	Async calls to Gamma and CLOB APIs; py-clob-client optional, not required for read-only
Frontend	React 18 + Vite	Ship source AND a pre-built dist/ served by FastAPI, so install stays simple but the client can extend
Charts	Recharts	LineChart with Brush (zoom) and Tooltip (hover)
Styling	Inline CSS-in-JS	No Tailwind, no UI framework — white background, dependency-light, per client request
Process manager	systemd	Unit file included; restart=always, starts on boot
CSV export	Python stdlib csv	Streamed from SQLite; Pandas not required

Everything is open source with no paid dependencies and no Docker, matching the client's constraints exactly.

3. Data Model (SQLite)

The schema uses a generic outcome model: an event contains markets (props), a market contains two or more outcomes, and prices are recorded per outcome. This single shape covers YES/NO, Team A/Team B, Win/Draw/Lose, and totals (Over/Under) without special cases. On Polymarket, multi-outcome situations are represented as groups of linked binary markets under one event; each outcome maps to one CLOB token ID.

```
CREATE TABLE events (
  id            INTEGER PRIMARY KEY,
  slug          TEXT UNIQUE NOT NULL,      -- polymarket event slug
  title         TEXT NOT NULL,
  created_at    TEXT NOT NULL DEFAULT (datetime('now'))
);

CREATE TABLE markets (
  id            INTEGER PRIMARY KEY,
  event_id      INTEGER NOT NULL REFERENCES events(id),
  condition_id  TEXT UNIQUE NOT NULL,      -- polymarket condition id
  question      TEXT NOT NULL,             -- e.g. "Match winner (full time)"
  kind          TEXT NOT NULL,             -- "yes_no" | "team" | "wdl" | "totals"
  tracking      INTEGER NOT NULL DEFAULT 0,
  poll_interval INTEGER NOT NULL DEFAULT 5, -- seconds, configurable
  created_at    TEXT NOT NULL DEFAULT (datetime('now'))
);

CREATE TABLE outcomes (
  id            INTEGER PRIMARY KEY,
  market_id     INTEGER NOT NULL REFERENCES markets(id),
  label         TEXT NOT NULL,             -- "Yes", "Arsenal", "Over 2.5", ...
  token_id      TEXT UNIQUE NOT NULL,      -- CLOB token id (from Gamma clobTokenIds)
  position      INTEGER NOT NULL           -- display order / chart color index
);

CREATE TABLE ticks (
  id            INTEGER PRIMARY KEY,
  outcome_id    INTEGER NOT NULL REFERENCES outcomes(id),
  ts            TEXT NOT NULL,             -- ISO-8601 UTC, set at poll time
  price         REAL NOT NULL              -- midpoint, 0..1
);
CREATE INDEX idx_ticks_outcome_ts ON ticks(outcome_id, ts);
```

Every poll inserts one row per outcome; nothing is aggregated or discarded — this was an explicit promise to the client. Store the midpoint price by default and document this in the README (best bid/ask could be added as extra columns in V2). Enable PRAGMA journal_mode=WAL so the FastAPI readers never block the collector's writes. Dashboard “database size” is `os.path.getsize` on `prices.db` (plus `-wal`).

4. Polymarket API Integration

All data the client needs is available from Polymarket's free public APIs with no authentication, no API key, and no wallet. Two services are used.

4.1 Gamma API — discovery (hit only when adding an event)

Base URL: <https://gamma-api.polymarket.com>. Given an event URL or slug pasted by the user, call `GET /events?slug=...` to fetch the event and all markets inside it. Each market carries its question, outcomes array, conditionId, and clobTokenIds — the token IDs are what the collector polls on the CLOB side. The add-event flow in the UI maps 1:1 to this call: show the returned markets as checkboxes, persist the ticked ones. Parse robustness note: outcomes and clobTokenIds are returned as JSON-encoded strings inside the JSON payload; `json.loads` them defensively.

4.2 CLOB API — prices (the polling loop)

Base URL: <https://clob.polymarket.com>. Read endpoints require no authentication. Use the batch midpoint endpoint so one HTTP request per poll cycle covers every tracked outcome:

```
POST /midpoints
Body: [{"token_id": "1234..."}, {"token_id": "5678..."}, ...]
-> {"1234...": "0.62", "5678...": "0.38", ...}

# Fallback per-token: GET /midpoint?token_id=...
# Order book if ever needed: POST /books
# V2 backfill: GET /prices-history?market=<token_id>&interval=...
```

The collector groups all tracked outcomes that share the same poll interval into one batch request, timestamps the response on arrival (UTC), and inserts the ticks in a single SQLite transaction.

4.3 Official rate limits (verified from docs.polymarket.com, July 2026)

Enforcement is Cloudflare throttling: requests over the limit are delayed and queued rather than immediately rejected, limits reset on sliding windows, and an eventual 429 carries no `Retry-After` header. The limits relevant to this application:

API	Endpoint	Limit
CLOB	/midpoint, /price, /book (single)	1,500 req / 10s
CLOB	/midpoints, /prices, /books (batch)	500 req / 10s
CLOB	/prices-history	1,000 req / 10s
CLOB	general	9,000 req / 10s
Gamma	/events	500 req / 10s
Gamma	/markets	300 req / 10s
Gamma	general	4,000 req / 10s
Global	all endpoints combined	15,000 req / 10s

Budget math: one batched poll every 3 seconds is ~3.3 requests per 10s window against a 500 req/10s allowance — under 1% utilisation regardless of how many markets are tracked. Rate limits are a non-issue for V1. Defensive code required anyway: exponential backoff with jitter on HTTP 429, generous client timeout (throttled-but-slow responses are successes, do not retry-storm them), and skip-and-log on repeated failure so one dead market never stalls the scheduler.

5. Backend Architecture

A single FastAPI application serves both the JSON API and the built frontend. APScheduler runs inside the same process (AsyncIOScheduler), one job per distinct poll interval. A systemd unit keeps the process alive 24/7 — this is what fulfils “collection continues even if the browser is closed”.

5.1 Suggested project layout

```
polymarket-tracker/
  app/
    main.py          # FastAPI app, mounts /api and serves frontend dist/
    api.py           # route handlers (thin)
    collector.py     # APScheduler jobs, batch polling, backoff
    polymarket.py    # Gamma + CLOB client (httpx), parsing helpers
    db.py            # SQLite connection, schema init, queries
    models.py        # pydantic response/request models
    config.py        # settings: default interval, db path, port
  frontend/         # Vite + React source (template already approved)
    src/App.jsx      # evolved from polymarket-tracker-dashboard.jsx
    dist/            # pre-built bundle, committed, served by FastAPI
  prices.db          # created on first run
  polymarket-tracker.service # systemd unit
  requirements.txt
  README.md          # install: python -m venv, pip install, systemd enable
```

5.2 REST API contract (what the frontend mocks map to)

Method + path	Purpose	Replaces mock
GET /api/dashboard	Stats: tracked/total counts, db size, last successful update, records today	refresh() stats
GET /api/markets	List all tracked markets with event, kind, outcomes, status, record count, last update, recent sparkline points	MOCK_MARKETS
POST /api/events/lookup	Body {url_or_slug} → event + its markets/props (Gamma passthrough, not yet persisted)	findEvent() / MOCK_EVENT_RESULT
POST /api/events/track	Body {slug, market_condition_ids[]} → persist event, selected markets, outcomes; start jobs	trackSelected()
POST /api/markets/{id}/start	Resume polling	toggle()
POST /api/markets/{id}/stop	Pause polling (data retained)	toggle()
GET /api/markets/{id}/ticks?limit=&before;=■=	Paged time-series rows for the chart and the ticks table	makeSeries()
GET /api/markets/{id}/export.csv	Streams full history: timestamp column + one price column per outcome	exportCsvFor()
PATCH /api/markets/{id}	Update poll_interval	—

CSV format (client-approved via the mockup): header row “timestamp, _price, ...”, ISO-8601 UTC timestamps, prices with three decimals, one row per poll, full history, oldest first.

5.3 systemd unit (ship as polymarket-tracker.service)

```
[Unit]
Description=Polymarket Price Tracker
After=network-online.target

[Service]
User=tracker
WorkingDirectory=/opt/polymarket-tracker
```

```
ExecStart=/opt/polymarket-tracker/venv/bin/uvicorn app.main:app --host 0.0.0.0 --port 8000
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

6. Frontend — Layout Specification (all pages)

The client has approved this layout from the delivered React mockup (polymarket-tracker-dashboard.jsx). Reproduce it faithfully; the mockup file is the visual source of truth and every mock function inside it is commented with its replacement API call. The app is a single-page application with two views plus one inline panel.

6.1 Design tokens

Token	Value	Used for
Background	#FFFFFF	Everything — explicit client requirement (plain white)
Ink	#1A1D23	Primary text, primary buttons
Sub / Faint	#6B7280 / #9CA3AF	Secondary text, metadata lines, placeholders
Border / Soft	#E5E7EB / #F8F9FB	1px borders, table header + hover backgrounds
Green / Red	#0E9F6E / #D64545	Tracking status, start/stop buttons, tick deltas
Series palette	#2563EB #D97706 #7C3AED #0E9F6E	Outcome chips, chart lines, table columns — assigned by outcome position, consistent across all views
Fonts	Inter (UI) / IBM Plex Mono (numbers)	Monospace with tabular numerals for every price, count and timestamp

Overall character: quiet data tool, not a marketing page. 8px border radius on cards, 1px borders, no shadows, no gradients. Row hover tint, subtle pulse animation on active tracking dots, spinner on refresh; all animation disabled under prefers-reduced-motion.

6.2 Global header (both views)

Full-width bar with 1px bottom border. Left: app name “Market Tracker” (bold 16px) followed by a mono caption “polymarket · price history · v1”. Right: a green pulsing dot with “collector running” (backend health from /api/dashboard), and a Refresh button with a rotating icon and disabled state while a fetch is in flight.

6.3 View 1 — Dashboard

Content column max-width 1100px, centred. Top to bottom:

Stats row. Four equal cards in a wrapping flex row: Tracked markets (“active / total”), Database size (hint “prices.db (SQLite)”), Last successful update, Records today. Uppercase 11px label, 21px mono value, small faint hint line.

Add-event bar. One mono-font text input (placeholder “Paste a Polymarket event URL or ID...”) plus a dark primary button labelled “Find event”; Enter key submits. Submitting calls POST /api/events/lookup and opens the event panel below.

Event panel (inline, soft-grey card). Caption “EVENT FOUND”, event title, and the line “This event contains N markets (props). Tick the ones to track — each is stored as its own price series”. Then one white checkbox row per market: checkbox, market name (e.g. “Total goals — Over/Under 2.5”), kind label, and right-aligned colored outcome chips. Footer: primary button “Track selected (n)” (disabled at n=0, calls POST /api/events/track) and a ghost Cancel button. This panel is how the client’s win/draw/lose vs totals question is answered in the UI.

Market list. A bordered card of CSS-grid rows (not an HTML table — this fixed an earlier button-cropping bug). Grid template: minmax(240px,1.6fr) 110px minmax(140px,0.9fr) 100px 210px with column headers MARKET, TREND, OUTCOMES, RECORDS, and an unlabelled actions column. Per row: (1) status dot + “Event · Question” on one ellipsized line with full text in the title attribute, and a mono metadata line underneath — id · kind · every Ns · updated X ago; (2) a 96×26 inline SVG sparkline of the leading outcome, dimmed at 35% opacity when stopped; (3) colored outcome chips; (4) mono record count; (5) three right-aligned buttons that must never crop: Start

(green) / Stop (red outline), “■ CSV” (direct download, no navigation), “History →”. Below the card, the faint line “Data collection runs server-side and continues when this page is closed.”

6.4 View 2 — Market History

Opened via History →. Header block: small “← Back to markets” ghost button with the mono market id beside it; uppercase faint event name; 20px market question; sub-line “kind · polled every Ns · N records stored”.

Chart card. Bordered card containing a Recharts LineChart, height 360, one line per outcome using the series palette in outcome order, no dots, width 1.8. Y-axis fixed 0–1 with two-decimal mono ticks; X-axis mono time labels. Tooltip on hover shows timestamp plus every outcome's price at three decimals (the client's hover requirement). A Recharts Brush strip (height 26) under the plot provides drag-to-zoom (the zoom requirement). Caption inside the card explains both gestures.

Action row. “■ Export CSV” (primary, dark — the client demanded this button be obvious), “Refresh data” with spinner, and Start/Stop tracking toggle.

Recorded ticks table (the client's “Time-Series Chronological Table”). Heading “Recorded ticks” with the caption “one row per poll (Ns)” and a right-aligned order toggle “Newest first ↓ / Oldest first ↑”. Bordered, vertically scrolling container (max-height 340) with a sticky header row: TIMESTAMP left-aligned, then one right-aligned column per outcome, each header tinted with that outcome's series color. Every cell is mono with tabular numerals; each price carries a tiny ▲ (green) or ▼ (red) delta indicator against the chronologically previous tick, and a faint · when unchanged. In production this table pages through the full stored history via GET /api/markets/{id}/ticks; the footer line states that every poll is stored and nothing is thrown away.

6.5 States and behaviours

Refresh: disabled + spinning icon while in flight; on completion, record counts and “last update” fields update. Empty market list: keep the stats and add-event bar; show a short faint hint in the list card. New market rows start at 0 records, tracking on, “updated just now”. Buttons show a soft background on hover and a visible focus ring. Long market names never push the actions column off-screen — they truncate. All number-bearing text uses the mono font so columns align.

7. Delivery Checklist

Definition of done for V1, assembled from the job post and every client message: the collector polls each tracked market at its configured interval and inserts one timestamped row per outcome into SQLite; polling survives browser close, SSH logout, and server reboot (systemd); the dashboard shows tracked-market count, database size, last successful update and per-market status; events are added by URL/ID with prop selection covering YES/NO, Team A/Team B, Win/Draw/Lose and totals; every market has start/stop, an interactive zoom-and-hover chart, the chronological ticks table, and a working CSV export reachable from both the list row and the history view; the code is modular and commented; the README covers installation on a fresh Ubuntu server (venv, pip install -r requirements.txt, systemd enable, default port) in copy-paste form; no paid libraries, no Docker.

Suggested build order for Claude Code: (1) db.py schema + polymarket.py Gamma/CLOB client with tests against live endpoints; (2) collector.py batch polling loop with backoff; (3) FastAPI routes per the contract in 5.2; (4) port the approved mockup into frontend/src, replacing each marked mock with its fetch call; (5) build dist/, wire static serving; (6) systemd unit + README; (7) end-to-end test: add a live football event, track the winner and one totals prop for ten minutes, verify chart, table order toggle, deltas, and CSV contents.

Reference artifact: polymarket-tracker-dashboard.jsx — the approved mockup. Mock boundaries inside it: MOCK_MARKETS and MOCK_EVENT_RESULT (data), refresh(), findEvent(), trackSelected(), exportCsvFor(), makeSeries() (behaviour). Each is commented with the endpoint that replaces it.